



Penetration Testing

Talisman Keyring (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
Audit Analysis and Review	14
Conclusion	16
Our Methodology	17
Disclaimers	19
About Hashlock	20

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Talisman team partnered with Hashlock to conduct a security Penetration Test of their Talisman Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and the community that the deployed tools were secure.

Project Context

Talisman is a crypto wallet and asset management dashboard that unlocks a new world of web3 applications for Ethereum and Polkadot.

The Talisman browser extension allows you to safely store, send, receive, and harness your assets across relaychains, parachains and everything EVM. The Talisman web app allows you to seamlessly manage your assets in greater detail. Keep track of all your NFTs across all chains and explore new dApps and interesting projects.

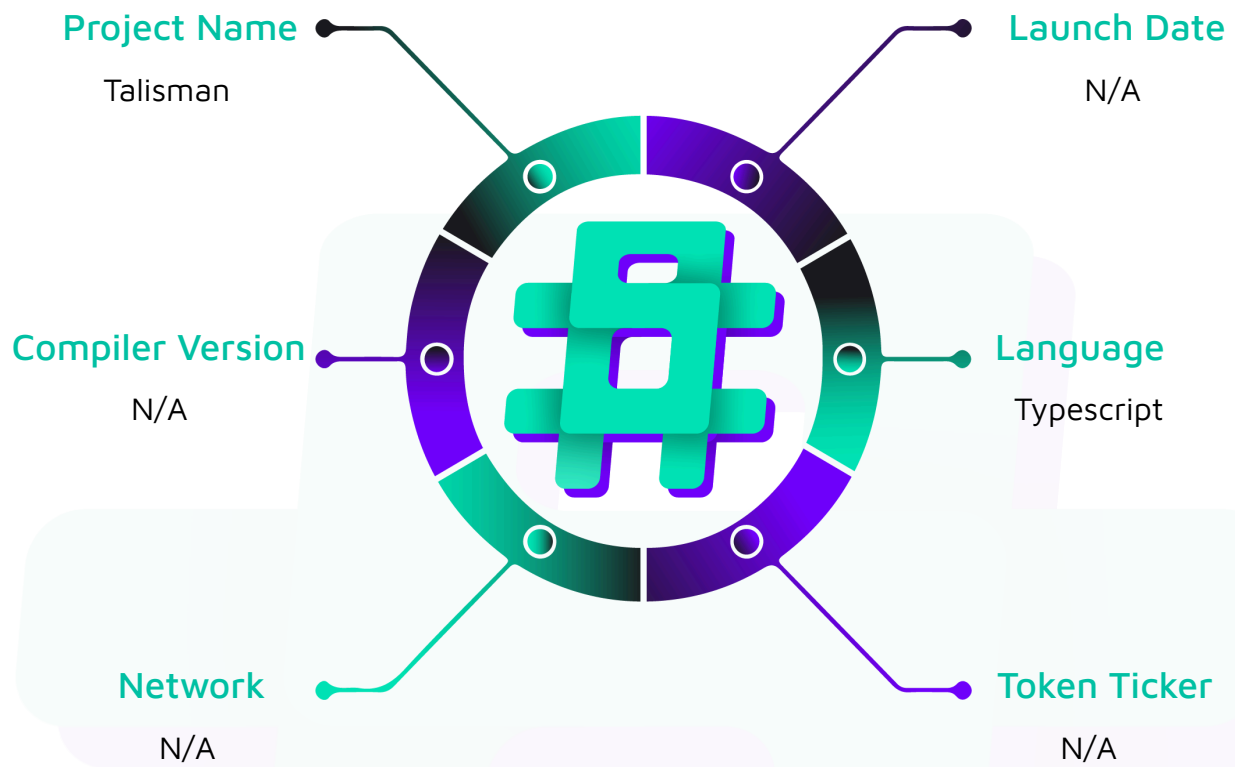
Project Name: Talisman

Project Type: Keyring

Website: <https://app.talisman.xyz/>

Logo:



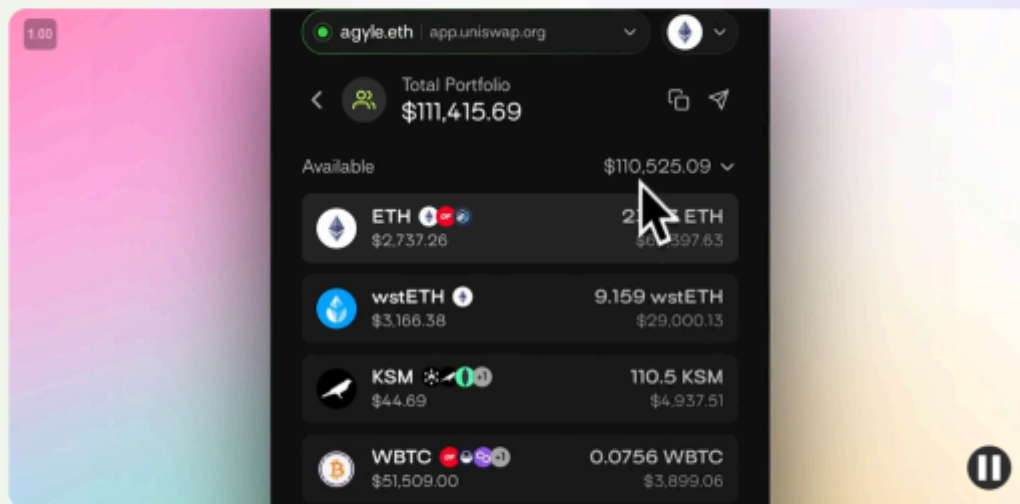
Visualised Context:

Project Visuals:

Multi-Chain Made Easy

Talisman is an Ethereum and Polkadot wallet that makes web3 simple for beginners and unlocks superpowers for pros.

Download Wallet



Try the Talisman wallet

Available on all good browsers



Add to Browser

Works with External Devices

Talisman works seamlessly with your favourite external devices like Ledger and Polkadot Vault.



Stake ETH, DOT, and More

With just a few clicks you can easily earn passive income on supported assets like: DOT, ETH, TAO, ASTR, and more.



#hashlock.

Hashlock Pty Ltd

Penetration Test scope

At Hashlock, we executed a comprehensive penetration test on the Talisman project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Our assessment addressed the client's specified security concerns regarding the Talisman project.

Description	Talisman Project code base
Platform	Typescript
Penetration Test Date	February/March 2025
Repository	https://github.com/TalismanSociety/talisman/pull/1811/
Package 1	packages/crypto
Package 2	packages/keyring
Package 3	packages/extension-core/domains/keyring

The goals for auditing the referred packages can be encountered in the [Audit Analysis and Review](#) section and the resolution following a comprehensive audit of the specified scope and objectives is presented in the [Conclusion](#) section. It outlines the results and addresses the concerns raised by the Talisman team.

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering. We initially identified some vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Low-severity vulnerabilities

2 QAs

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Low

[L-01] Keyring - Dependencies not pinned to exact versions

Description

The application uses external dependencies with caret ranges (^x.x.x) instead of exact version pinning. In packages/crypto/package.json. This approach can potentially allow for dependency-based supply chain attacks if a malicious version is published within the compatible range.

Recommendation

Pin all external dependencies to exact versions (x.x.x) instead of using compatible version ranges (^x.x.x) to mitigate the risk of dependency-based attacks. This is particularly important for cryptographic libraries that handle sensitive operations.

Status

Resolved

QA

[Q-01] Keyring - Unresolved TODO comments in codebase

Description

Several TODO comments remain in the codebase, indicating incomplete or unresolved implementation details. Notable examples include missing schema validation in `Keyring.load` and unresolved questions about payload handling in the signing module.

Here's a list of all the TODOs found in the provided code files:

- `packages/keyring/src/keyring/Keyring.ts`

```
// TODO: schema check ? (in the load static method)
```

- `packages/crypto/src/signing/index.ts`

```
// TODO determine if we need to hash payloads before ecdsa signing
```

```
// TODO determine if we need to wrap polkadot payloads with <Bytes></Bytes>
```

```
// TODO pjs signs blake2 hashes - should we handle this? (in the ecdsa case)
```

```
// TODO pjs signs keccak hashes - should we handle this? (in the ethereum case)
```

```
// TODO pad payload with <Bytes></Bytes> for polkadot checks ? (in the  
verifySignature function)
```

- `packages/crypto/index.ts`

```
// @dev: dont export signing module until necessary, wallet doesn't use it yet
```

Recommendation

Address the TODO items, especially operations like schema validation and payload handling for signatures.

Status

Acknowledged

[Q-02] Keyring - Display of error messages to users

Description

In `apps/extension/src/ui/domains/System/MigrationProgress.tsx`, the component directly displays raw error messages to users through `{migration.errors.map((err, idx) => (<li key={idx} className="list-disc">{err}</li>))}`. This could potentially expose unwanted errors to users. However, since these errors are client-side, they do not pose a direct security risk but should rather be considered an UI/UX issue.

Recommendation

Implement user-friendly error messages or error codes instead of displaying raw technical errors. Consider implementing a mapping of technical errors to user-friendly descriptions to improve user experience.

Status

Acknowledged

Audit Analysis and Review

In order to ensure the continued security and robustness of the Talisman keyring implementation, our audit addressed the following specific goals:

Keyring and Derivation Safety

Our team verified that the keyring generation and derivation processes are implemented securely, ensuring that key material is generated and managed without introducing vulnerabilities.

Encryption and Decryption Integrity

The analysis confirmed that encryption and decryption routines, along with their underlying cryptographic libraries, are correctly implemented and adhere to industry best practices.

Brute Force Resilience

Hashlock evaluated the resistance of the new keyring against brute force attacks, ensuring that its security level is at least equivalent to that of the previous implementation.

Dependency Vulnerability Assessment

All external dependencies were reviewed for potential vulnerabilities. Particular attention was given to ensuring that any unpinned or loosely defined dependency versions do not expose the system to supply chain attacks.

Secure Key Encryption

The audit verified that keys are properly encrypted and stored securely. This ensures that sensitive key material remains protected and is not inadvertently exposed.

Check Password Method Analysis

Hashlock assessed the newly implemented `checkPassword` method to confirm that it does not compromise the overall security of the keyring. Specifically, the audit ensured that even if an attacker (e.g., "Alice") knows the decrypted data (such as the



`PASSWORD_CHECK_PHRASE`), this knowledge does not enable them to reverse engineer the original encryption key and thereby decrypt the remainder of the keyring.

Review of Keyring Method Callers

Finally, a detailed review was conducted on the callers of methods within the `packages/extension-core/domains/keyring` module. This step ensured that these methods, which are critical for manipulating or reading secret keys and mnemonics, are invoked in a secure context without introducing additional risks.

Conclusion

After Hashlock's analysis, the Talisman project seems to have a sound and well-tested code base. The new keyring implementation was evaluated and confirmed to maintain equivalent brute force resistance compared to the previous implementation. A comprehensive audit of the micro-sr25519 library verified its security compliance and ensured it can still be considered as secure. The password verification method was assessed, and appropriate cryptographic techniques were determined. Encryption mechanisms were found to implement industry standard protocols with proper parameter selection. As per standard practice, external dependencies were examined for proper version control while detailed dependency audits remained outside the assessment scope. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.